

Day 40 – Data Preprocessing in Machine Learning

Before we can train a Machine Learning model, the raw dataset must be cleaned and prepared. This step is called **Data Preprocessing**, and it ensures that the data is consistent, complete, and ready for learning.

Why is Data Preprocessing Important?

- Real-world datasets are rarely perfect. They often contain:
 - Missing values
 - Categorical (non-numeric) features
 - Inconsistent formats or scales
- ML models work best when input data is:
 - Numeric
 - Clean (no NaN values)
 - Properly scaled

Key Steps in Data Preprocessing

1. Import the dataset
 2. Separate independent (features) and dependent (target) variables
 3. Handle missing data
 4. Encode categorical variables
 5. Split into training and test sets
 6. Apply feature scaling
-

Importing Libraries

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Importing the Dataset

We start by loading our dataset using `pandas.read_csv()`.

```
In [2]: data = pd.read_csv(r"C:\Users\Arman\Downloads\dataset\data.csv")
```

```
In [3]: print("Dataset Info:")
data.info()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   State       10 non-null    object
1   Age         9 non-null     float64
2   Salary      9 non-null     float64
3   Purchased   10 non-null    object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
In [4]: print("Missing Values:")
data.isnull().sum()
```

Missing Values:

```
Out[4]: State      0
Age          1
Salary       1
Purchased    0
dtype: int64
```

```
In [5]: data
```

```
Out[5]:
```

	State	Age	Salary	Purchased
0	Mumbai	44.0	72000.0	No
1	Bangalore	27.0	48000.0	Yes
2	Hyderabad	30.0	54000.0	No
3	Bangalore	38.0	61000.0	No
4	Hyderabad	40.0	NaN	Yes
5	Mumbai	35.0	58000.0	Yes
6	Bangalore	NaN	52000.0	No
7	Mumbai	48.0	79000.0	Yes
8	Hyderabad	50.0	83000.0	No
9	Mumbai	37.0	67000.0	Yes

Separating Independent (x) and Dependent (y) Variables

Features vs Target

- **Independent variables (x):** All input features (columns except the target).
- **Dependent variable (y):** The target column we want to predict.

```
In [6]: x = data.iloc[:, :-1].values  # All columns except last
y = data.iloc[:, 3].values          # Last column
```

```
In [7]: print(x)

[['Mumbai' 44.0 72000.0]
 ['Bangalore' 27.0 48000.0]
 ['Hyderabad' 30.0 54000.0]
 ['Bangalore' 38.0 61000.0]
 ['Hyderabad' 40.0 nan]
 ['Mumbai' 35.0 58000.0]
 ['Bangalore' nan 52000.0]
 ['Mumbai' 48.0 79000.0]
 ['Hyderabad' 50.0 83000.0]
 ['Mumbai' 37.0 67000.0]]
```

```
In [8]: print(y)

['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

Handling Missing Data

Why Handle Missing Data?

- Real-world datasets often have missing values.
- Many ML algorithms cannot work with missing data directly.
- A common strategy is to **replace missing values with the mean** of the column.

```
In [9]: from sklearn.impute import SimpleImputer

# Imputer: replace missing numeric values with column mean
imputer = SimpleImputer()

# Fit only on numeric columns (assuming Age, Salary are cols 1 and 2)
imputer = imputer.fit(x[:,1:3])

# Transform the dataset
x[:,1:3] = imputer.transform(x[:,1:3])
```

```
In [10]: print(x)

[['Mumbai' 44.0 72000.0]
 ['Bangalore' 27.0 48000.0]
 ['Hyderabad' 30.0 54000.0]
 ['Bangalore' 38.0 61000.0]
 ['Hyderabad' 40.0 63777.77777777778]
 ['Mumbai' 35.0 58000.0]
 ['Bangalore' 38.77777777777778 52000.0]
 ['Mumbai' 48.0 79000.0]
 ['Hyderabad' 50.0 83000.0]
 ['Mumbai' 37.0 67000.0]]
```

```
In [11]: print("Before Imputation:")
print(data)

print("\nAfter Imputation:")
print(x)
```

Before Imputation:

	State	Age	Salary	Purchased
0	Mumbai	44.0	72000.0	No
1	Bangalore	27.0	48000.0	Yes
2	Hyderabad	30.0	54000.0	No
3	Bangalore	38.0	61000.0	No
4	Hyderabad	40.0	NaN	Yes
5	Mumbai	35.0	58000.0	Yes
6	Bangalore	NaN	52000.0	No
7	Mumbai	48.0	79000.0	Yes
8	Hyderabad	50.0	83000.0	No
9	Mumbai	37.0	67000.0	Yes

After Imputation:

```
[[ 'Mumbai' 44.0 72000.0]
 [ 'Bangalore' 27.0 48000.0]
 [ 'Hyderabad' 30.0 54000.0]
 [ 'Bangalore' 38.0 61000.0]
 [ 'Hyderabad' 40.0 63777.77777777778]
 [ 'Mumbai' 35.0 58000.0]
 [ 'Bangalore' 38.77777777777778 52000.0]
 [ 'Mumbai' 48.0 79000.0]
 [ 'Hyderabad' 50.0 83000.0]
 [ 'Mumbai' 37.0 67000.0]]
```

Encoding Categorical Data

Why Encode?

Machine learning models work with numbers, not text.
Categorical data (like State, Purchased) must be converted to numeric form.

Two common methods:

- **Label Encoding:** Assigns a number (0,1,2,...) to each category.
- **One-Hot Encoding:** Creates separate binary columns for each category.

In this example, I used **Label Encoding**.

```
In [12]: from sklearn.preprocessing import LabelEncoder

# Encode first column of X (categorical independent variable)
labelencoder_x = LabelEncoder()
x[:,0] = labelencoder_x.fit_transform(x[:,0])

# Encode dependent variable y
labelencoder_y = LabelEncoder()
y = labelencoder_y.fit_transform(y)
```

```
In [13]: print("Before Encoding:")
print(data)

print("\nAfter Encoding:")
print("Encoded x:\n", x)
print("Encoded y:\n", y)
```

Before Encoding:

	State	Age	Salary	Purchased
0	Mumbai	44.0	72000.0	No
1	Bangalore	27.0	48000.0	Yes
2	Hyderabad	30.0	54000.0	No
3	Bangalore	38.0	61000.0	No
4	Hyderabad	40.0	NaN	Yes
5	Mumbai	35.0	58000.0	Yes
6	Bangalore	NaN	52000.0	No
7	Mumbai	48.0	79000.0	Yes
8	Hyderabad	50.0	83000.0	No
9	Mumbai	37.0	67000.0	Yes

After Encoding:

Encoded x:

```
[[2 44.0 72000.0]
 [0 27.0 48000.0]
 [1 30.0 54000.0]
 [0 38.0 61000.0]
 [1 40.0 63777.77777777778]
 [2 35.0 58000.0]
 [0 38.77777777777778 52000.0]
 [2 48.0 79000.0]
 [1 50.0 83000.0]
 [2 37.0 67000.0]]
```

Encoded y:

```
[0 1 0 0 1 1 0 1 0 1]
```

Splitting the Dataset into Training and Test Sets

Why Split?

We split our dataset to evaluate performance:

- **Training set** → used to train the model.
- **Test set** → used to evaluate accuracy on unseen data.

```
In [14]: from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 0.2, random_state=0)

In [15]: print("x_train Length:", len(x_train))
print("x_test Length:", len(x_test))
print("y_train Length:", len(y_train))
print("y_test Length:", len(y_test))
```

```
x_train Length: 8
x_test Length: 2
y_train Length: 8
y_test Length: 2
```

Summary – Data Preprocessing in ML

In this notebook, I prepared raw data for machine learning by following a structured preprocessing pipeline:

1. **Imported the dataset** using `pandas` and checked its structure.
2. **Separated independent variables (x)** and the **dependent variable (y)**.
3. **Handled missing data** using `SimpleImputer` (mean replacement).
4. **Encoded categorical variables** with `LabelEncoder` to convert text into numeric values.
 - Note: For many problems, **One-Hot Encoding** is preferred to avoid imposing order.
5. **Split the dataset** into training and test sets using `train_test_split` to evaluate model performance fairly.

Key Takeaway:

Data preprocessing is the **first and most crucial step** in building ML models.

Clean, consistent, and properly formatted data ensures that models learn effectively and perform reliably.